COMP7610 SMART CONTRACT LAB ASSIGNMENT 3

HKU Drone Club on Sepolia

Ethereum Sepolia: chain ID 11155111

01 System overview and baseline requirements

The project combines an OpenZeppelin ERC-20 club-points token (CPT, 18 decimals) with an ERC-721 commemorative collection (Campus-Memo). Deployment creates exactly 1,000 CPT in the CPT contract treasury. Only club administrators may mint or distribute points. NFT issuance is capped at ten; each issuance burns 100 CPT from the issuing administrator. Members receive NFTs, while owners can subsequently transfer or sell them.

02 Additional functions and design decisions

Membership and welcome	`applyForMembership` records an application; administrator-only `approveMembership` grants membership and transfers 5 CPT from the treasury.
Voluntary withdrawal	`withdrawFromClub` burns the member's entire current CPT balance and revokes membership. Existing NFTs remain with their owner.
Administrative burn	`adminBurnPoints` destroys a specified user balance amount without allowance; both the balance and total supply decrease.
Treasury recovery	`adminRecoverPoints` returns user CPT to the treasury without allowance. Supply, membership and NFT ownership remain unchanged.
CPT user actions	Standard `transfer`, `approve` and `transferFrom` support direct or delegated transfers, including after withdrawal. Holders can also use `burn` or approve `burnFrom`.
Controlled NFT issuance	`mintMemo(member)` checks CPT's current administrator role and the recipient's membership. The administrator approves and pays 100 CPT; the recipient is not charged. Roles can be granted or revoked through AccessControl.
NFT transfers and sales	Standard ERC-721 transfers and owner-approved delegation (`approve` / `setApprovalForAll`) remain available. `listMemo` escrows an NFT; `cancelMemoListing` returns it; `buyMemo` transfers approved CPT to the seller and the NFT to the buyer. Former members may still trade.
Stable metadata	`tokenURI(id)` generates Base64 JSON on-chain with a numbered name and Edition attribute. One fixed GitHub image is shared; transfers and withdrawal do not alter metadata. No NFT burn or administrator recovery exists.

03 Deployment identifiers and public source

All addresses below link directly to Sepolia Etherscan. T and N denote the deployed contracts; A, B and C denote the test participants.

T: CPT	`0x2cfc2b3Ed989ef5bA52E586dbcB5A0D8556d96C9`
N: CampusMemo	`0x85C5E2da4e940420f3a429D482C22763345e16b9`
A: administrator	`0x861BCc126C001A3313F58436c7629a1B168883aE`
B: member / seller	`0x136bD9945a73Ba50A060Ea4362f33ede7727332d`
C: member / buyer	`0xa66AebD84583dc72F4803c22EbC4fB34b1908144`

Open-source code repository: `https://github.com/01orbit/COMP7610-Assignment3`

Image (fixed commit):
`https://raw.githubusercontent.com/01orbit/drone-nft-picture/2c2f2b64bbacf9f201216d6a8e9ec9e557df650f/drone-nft.png`

04 Deployment procedure and creation evidence

1. Publish the image above, then prepare Remix copies with `npm run remix:prepare`. Compile CPT and CampusMemo in web Remix: Solidity 0.8.24, OpenZeppelin 5.0.2, optimizer 200 runs, EVM paris; enable Generate contract metadata.
2. Select Remix VM and run `scripts/remix-test.js` for the full functional checks. Switch to Browser Extension / MetaMask, connect A on Sepolia, and keep transaction Value at zero.
3. Deploy CPT with `initialAdmin=A`: CPT creation transaction. Deploy CampusMemo with T and the fixed image URL: CampusMemo creation transaction. Each linked receipt identifies A as creator and T or N as the created contract.
4. Initial reads: `balanceOf(T)=totalSupply()=1000 CPT`, `WELCOME_POINTS()=5 CPT`; N links to T, costs 100 CPT per mint, caps supply at 10 and initially has no NFTs. The Sepolia checker `scripts/remix-sepolia-check.js` in `initial` mode confirms these values without sending transactions.

Sepolia verification walkthrough

Use the linked addresses on page 1 to locate transactions to T or N, ordered by block and transaction index. Open each receipt; check Success, From, To, Input Data and Logs. Values below are in whole CPT; in calldata, $u = 10^{18}$ base units (e.g. 80 CPT = `80e18` in Remix).

05 Function-test sequence and verification checkpoints

Step	Sender, contract and call	Evidence to check in receipt / state
1	B: `T.applyForMembership()`; A: `T.approveMembership(B)`	Application then approval events; treasury transfers 5 CPT to B. B becomes a member; treasury 995.
2	C: `T.applyForMembership()`; A: `T.approveMembership(C)`	C receives 5 CPT and membership; treasury 990; total supply still 1,000.
3	A: `T.adminRecoverPoints(B, 2u)`	`PointsRecoveredByAdmin`; transfer B to T of 2 CPT. B 3; treasury 992; supply unchanged.
4	A: `T.distributePoints(A, 100u)`	Transfer T to A of 100 CPT; A 100; treasury 892.
5	A: `T.approve(N, 100u)`	`Approval(A,N,100u)`; no balance change.
6	A: `N.mintMemo(B)`	CPT burn A to zero of 100; NFT mint zero to B, ID 1; `MemoMinted`. A 0; B 3; supply 900.
7	B: `T.transfer(C, 1u)`	Transfer B to C of 1 CPT; B 2; C 6.
8	A: `T.distributePoints(C, 74u)`	Transfer T to C of 74 CPT; C 80; treasury 818.
9	B: `N.listMemo(1, 80u)`	`MemoListed`; NFT moves B to N (escrow), price 80 CPT.
10	B: `N.cancelMemoListing(1)`	`MemoListingCancelled`; NFT returns N to B; listing cleared.
11	B: `N.listMemo(1, 80u)`	NFT moves B to N again; seller B, price 80 CPT.
12	C: `T.approve(N, 80u)`	`Approval(C,N,80u)`; no payment yet.
13	C: `N.buyMemo(1)`	`MemoPurchased`; CPT transfer C to B of 80; NFT N to C; listing cleared. B 82; C 0.
14	C: `N.safeTransferFrom(C, B, 1)`	ERC-721 `Transfer`: C to B, ID 1; CPT balances unchanged.
15	B: `T.withdrawFromClub()`	CPT burn B to zero of 82; `MemberWithdrawn`; B no longer a member. NFT stays with B.

06 How to verify without changing the contracts

Historical evidence. In T/N's Transactions tabs, match sender and function above, then inspect Logs. For `Transfer` logs, indexed topics 1/2 are from/to addresses: T stores the CPT amount in Data, whereas N stores the NFT ID in topic 3. Decode hex integers; divide CPT amounts by 10^{18}. A burn uses the zero address as recipient; the NFT mint uses it as sender. Failed simulations and read-only calls do not create receipts. The administrator-burn function and rejected permission / cap cases are covered by source and VM tests, rather than this Sepolia demonstration sequence.

Read-interface availability. If source verification is pending, Etherscan shows bytecode without a verified Read Contract interface. Transaction inputs and logs remain public; decode them with the submitted contract ABI. After source verification, use Contract → Read Contract for the queries below. Alternatively compile the submitted source in Remix, load T/N using At Address, connect Sepolia, and perform only read calls. No administrator key is needed for reads.

Final checks. After exactly steps 1–15, `T.totalSupply()` and `T.balanceOf(T)` should each be 818 CPT; A/B/C each hold zero CPT. `T.isMember(B)=false`, `T.isMember(C)=true`, `N.totalMinted()=1`, `N.ownerOf(1)=B`, and `N.listings(1)` is cleared. Run `scripts/remix-sepolia-check.js` with the page-1 addresses, image URL and `stage: "after-demo"` for a read-only block snapshot. Use `inspect` if subsequent transactions changed the state.

Metadata check. Read `N.tokenURI(1)` and decode its Base64 JSON: name `Campus-Memo ##1`, Edition 1, and the page-1 image URL. Ownership changes and membership withdrawal leave that URI unchanged. The complete URI for token 1 is reproduced below.

When Input Data is not decoded: the first four bytes identify the method; subsequent ABI words contain the arguments. Match these selectors with the calls on page 2:

`applyForMembership`	`5e1ee52a`	`approveMembership`	`79a68cf1`
`adminRecoverPoints`	`6f1fa741`	`distributePoints`	`51a40e96`
`approve`	`095ea7b3`	`mintMemo`	`a932e4eb`
`transfer`	`a9059cbb`	`listMemo`	`b206477e`
`cancelMemoListing`	`a595af6a`	`buyMemo`	`88d7d663`
`safeTransferFrom` (3 args)	`42842e0e`	`withdrawFromClub`	`aa5ae8b7`

07 NFT URI (token 1)


```
data:application/json;base64,eyJuYW1lIjoiQ2FtcHVzLU1lbW8gIzEiLCJkZXNjcmlwdGlvbiI6IkEgbGltaXRlZC11ZGl0aW9uIGNvbW1lbW9yYXRpdmUgTkZUIG9mIHRoZSBIS1UgRHJvbmUgQ2x1Yi4iLCJpbWFnZSI6Imh0dHBzOi8vcmF3LmdpdGh1YnVzZXJjb250ZW50LmNvbS8wMW9yYml0L2Ryb25lLW5mdC1waWN0dXJlLzJjMmYyYjY0YmJhY2Y5ZjIwMTIxNmQ2YThlOWVjOWU1NTdkZjY1MGYvZHJvbmUtbmZ0LnBuZyIsImF0dHJpYnV0ZXMiOlt7InRyYWl0X3R5cGUiOiJFZGl0aW9uIiwidmFsdWUiOjF9XX0=
```